

Testing (pruebas) en React con Vitest + Testing Library

Para la realización de pruebas unitarias en un proyecto React realizado con Vite se usa la librería Vitest.

Para la realización de pruebas sobre componentes y de interacción con el usuario necesitamos además las librerías testing library de react y la librería jsdom.

1. Instalación y configuración de Vitest

Vitest es un framework de testing diseñado para complementar a Vite. Sirve para crear, ejecutar y estructurar las pruebas. Vitest se encarga de buscar los ficheros de test o pruebas que nosotros creemos, los ejecuta y devuelve los resultados de las pruebas. Para ello, cuando creamos un fichero de prueba de una función o componente, debemos nombrarlo igual que la función o componente seguido de .test.js o .test.jsx

Para poder usarlo debemos instalarlo. Abrimos una terminal (puede ser la del Visual Studio) y nos situamos dentro de la carpeta de nuestro proyecto **pruebas**:

npm install -D vitest

Con esto estamos instalando vitest como dependencia de desarrollo (-D).

Para poder usarlo debemos añadir la siguiente línea:

```
"test": "vitest"
```

al fichero package.json:

```
"scripts": {  
  "dev": "vite",  
  "build": "vite build",  
  "lint": "eslint .",  
  "preview": "vite preview",  
  "test": "vitest"  
},
```

2.Instalación de librerías Testing Library de React

Nos permiten renderizar nuestros componentes sobre el Virtual DOM sin necesidad de usar el navegador. Además, dispone de las herramientas necesarias para acceder e interactuar con el Virtual DOM.

Esta librería necesita de otraa herramienta para poder cargar las pruebas, ejecutarlas y comprobar los tests. Estas herramientas son las librerías jest-dom (del testing library) y jsdom.

Por lo tanto debemos instalar las siguientes librerías:

npm install -D @testing-library/react

npm install -D @testing-library/user-event

```
npm install -D @testing-library/jest-dom
```

```
npm install -D jsdom
```

Comprobamos su instalación en el package.json:

```
"devDependencies": {
  "@eslint/js": "^9.15.0",
  "@testing-library/jest-dom": "^6.6.3",
  "@testing-library/react": "^16.0.1",
  "@testing-library/user-event": "^14.5.2",
  "@types/react": "^18.3.12",
  "@types/react-dom": "^18.3.1",
  "@vitejs/plugin-react-swc": "^3.5.0",
  "eslint": "^9.15.0",
  "eslint-plugin-react": "^7.37.2",
  "eslint-plugin-react-hooks": "^5.0.0",
  "eslint-plugin-react-refresh": "^0.4.14",
  "globals": "^15.12.0",
  "jsdom": "^25.0.1",
  "vite": "^6.0.1",
  "vitest": "^2.1.8"
```

3. Configuración de ficheros para poder ejecutar los tests

Nos vamos al fichero de configuración de vite: vite.config.js

El fichero por ahora está así:

```
testalumni > JS vite.config.js > ...
1  import { defineConfig } from 'vite'
2  import react from '@vitejs/plugin-react-swc'
3
4  // https://vite.dev/config/
5  export default defineConfig({
6    |   plugins: [react()],
7  })
8
```

Debemos añadirle lo siguiente:

```

testalumni > JS vite.config.js > ...
1  import { defineConfig } from 'vite'
2  import react from '@vitejs/plugin-react-swc'
3
4  // https://vite.dev/config/
5  export default defineConfig({
6    plugins: [react()],
7    test: {
8      globals: true,
9      environment: 'jsdom',
10     setupFiles: './src/setupTests.js',
11     css: true,
12   },
13 })
14

```

Y generar el fichero src/setupTests.js con la importación de la librería jest-dom:

```

testalumni > src > JS setupTests.js
1  import '@testing-library/jest-dom'

```

4. Estructura básica de un test

La estructura para realizar un test simple de una función que realiza la suma de dos números es la siguiente:

```

it('la suma de -2 y 5 debe dar 3', () => {
  //Llamada a la función que estamos testeando
  const result = sum(-2,5);
  //El resultado que esperamos obtener
  expect(result).toBe(3)
});

```

Podemos simplificarlo y hacer la llamada dentro de expect().

```

it('la suma de -2 y 5 debe dar 3', () => {
  expect(sum(-2,5)).toBe(3)
});

```

Cuando escribimos un test lo que queremos es comprobar que los valores cumplen ciertas condiciones. La función `expect()` da acceso a varios *matchers* o *marcadores* que permiten validar diferentes cosas. En el ejemplo de arriba usamos el marcador `toBe()`. Es decir, estamos sumando $-2 + 5$ y esperamos que el resultado sea (`toBe`) 3. Hay muchos marcadores. Los diferentes *matchers* de la librería Vitest los puedes encontrar listados aquí:

<https://vitest.dev/api/expect.html>

El `describe()` se usa para agrupar diferentes tests dentro de él. Podemos también usar `describe()` dentro de otro `describe()`.

```
describe('comentario', () => {
  it('comentario', () => {
    //aquí va el test
  });
});
```

Si queremos poner varios test dentro de un `describe`:

```
describe('comentario', () => {
  it('test1', () => {
    //aquí va el test1
  });

  it('test2', () => {
    //aquí va el test2
  });
});
```

5. Cómo ejecutar los tests

Para ejecutar los tests hay que escribir el siguiente comando:

npm run test

Se ejecutarán los ficheros que estén nombrados con esta terminación: `.test.jsx test.js`

Si queremos ejecutar un determinado fichero de test, por ejemplo `TextBox.test.js`:

npm run test --testPathPattern TextBox.test.js

Si queremos ejecutar un determinado test, lo buscamos por su comentario.

Ejemplo: en el fichero TextBox.test.js tenemos el test cuyo comentario es 'probar' (ejemplo: test('probar', ()=>{}))

npm run test -- -t 'prueba'

6. Testear el renderizado correcto de un componente o elemento

Para poder testear el correcto **renderizado de un componente** en el Virtual DOM tenemos que usar la librería React Testing Library.

Ejemplo para comprobar el renderizado de elementos en el Virtual DOM

Suponemos que tenemos una aplicación React+Vite en la que tenemos nuestro componente App en el fichero App.jsx. Suponemos que dentro de App.jsx mostramos en el navegador una etiqueta <h1> que pones Hola mundo.

Contenido de src/App.jsx

```
function App() {  
  return (  
    <>  
      <h1>Hola Mundo</h1>  
    </>  
  )  
}  
export default App
```

Contenido del fichero de test de App.jsx, que es src/tests/App.test.jsx:

```
//Importamos render y screen de la librería testing library  
import { render, screen } from "@testing-library/react";  
//Importamos el componente que queremos testear  
import App from "../App";  
  
it('prueba renderizado h1', () => {  
  //Renderizamos el componente App  
  render(<App />);  
  //Buscamos la etiqueta h1 con el selector getByRole  
  const cabh1 = screen.getByRole('heading', {level : 1})  
  //Comprobamos que la etiqueta esté en el documento  
  expect(cabh1).toBeInTheDocument();  
});
```

Encontrar elementos en el Virtual DOM

Para encontrar elementos en el Virtual DOM, una vez que hemos renderizado nuestro componente, usamos la función `screen` seguida de un selector (*query*).

En los siguientes enlaces se muestran los tipos de queries que tiene la librería React Testing Library así como la guía de prioridad de su uso:

<https://testing-library.com/docs/queries/about>

<https://testing-library.com/docs/queries/about/#priority>

Nosotros vamos a ver en principio: `getByRole()` y `getByLabelText()`

Ejemplos de queries según el elemento a buscar			
Imagen o avatar	getByRole()		screen.getByRole('img')
		<Avatar src='/gato.png' />	screen.getByRole('img')
Campo de texto	getByLabelText()	<TextField label='Nombre' />	screen.getByLabelText('Nombre')
Cabecera (h1, h2, h3, ...)	getByRole()	Cabecera h3	screen.getByRole('heading', {level : 3})
Botón	getByRole()	<Button > Enviar </Button>	screen.getByRole('button', name: 'Enviar')

Ejercicio guiado que se hace en clase

Crea un proyecto react (JavaScript + SWC) e instala las librerías necesarias para poder realizar tests. Ejecútalo y revisa el documento `App.jsx`, verás que se renderizan dos imágenes, una etiqueta `<h1>` y un botón entre otras cosas.

Crea una carpeta `src/tests` y dentro crea el fichero `App.test.jsx`.

Haz los tests de renderizado de la etiqueta `h1`, el botón y las imágenes usando las *queries* (selectores). Ejecuta los tests.

7. Realización de pruebas unitarias, de renderizado de elementos y de interacción con el usuario

En esta sección veremos ejemplos de pruebas unitarias, seguiremos viendo ejemplos de cómo testear renderizado de elementos/componentes y también veremos ejemplos de como testear la interacción del usuario con los elementos del DOM.

Ejemplo 1: testear funciones (pruebas unitarias)

En este ejemplo (descargar código del campus) vamos a testear funciones, que es la entidad mínima a testear (pruebas unitarias). Estas funciones serán parte de un componente. En el ejemplo se crea el componente `src/components/MiComponente.jsx` que usará unas cuantas funciones que son las que vamos a testear. Estas funciones son: **`divide.js`**, **`sum.js`**, **`titleCase.js`** y **`isapple.js`**.

En las pruebas unitarias solo tenemos que ver si los parámetros de salida generados a partir de unos parámetro de entrada concuerdan con los esperados.

Para testear hemos creado una carpeta llamada `tests` y dentro creamos un fichero de test por cada función que vayamos a testear.

Ejemplo para hacer los tests de la función `sum`: la idea es hacer varios test a la función `sum`. Para ello los agrupamos dentro de una estructura describe:

```
describe('comentario', () => {
  it('comentario test 1', () => {
    //aquí va el test 1 de la sum
  });
  it('comentario test 2', () => {
    //aquí va el test 2 de la sum
  });
//Y todos los test que queramos hacer...
});

describe('Tests de la función sum', () => {
  it('la suma devuelve un número', () => {
    //Llamada a la función que estamos testeando
    const result = sum(2,3);
    //El resultado que esperamos obtener
    expect(typeof result).toBe('number')
  });
  it('la suma de 2 y 3 es 5', () => {
    //Llamada a la función que estamos testeando
    const result = sum(2,3);
    //El resultado que esperamos obtener
    expect(result).toBe(5)
  });
  // Podemos seguir poniendo más tests
});
```

Recuerda que los diferentes *matchers* de la librería Vitest los puedes encontrar listados aquí: <https://vitest.dev/api/expect.html>

Ejemplo 2: testear el renderizado correcto de un componente e interacciones con el usuario

En el ejemplo 1 hemos testado funciones que usamos en nuestros componentes: tests unitarios. En el ejemplo 2 testaremos el correcto **renderizado de un componente** en el DOM así como si queremos **simular la interacción del usuario con nuestra aplicación** (el usuario rellena un campo de texto, pica un botón, etc).

Ejemplo 2.1 Ejemplo de renderizado del componente TextBox.jsx

Vamos a testear el componente **TextBox.jsx** que lo que hace es mostrar en pantalla un `<Box>` de la librería `@MUI` con texto y un `<Button>`. Por defecto el `<Box>` tiene de color de fondo el *amarillo* (RGB(255,255,0)). Cuando el usuario pica el botón el color cambia a *rojo* (RGB(255,0,0)). Si el usuario vuelve a picar en el botón, el color cambia a *amarillo* ... y así sucesivamente.

Creamos un **fichero** **TextBox.test.jsx** donde escribiremos los test del componente en cuestión.

```
//importamos el componente para poder testearlo
import TextBox from "../components/TextBox";
//importamos render y screen: sirven para renderizar un componente y para obtener objetos del
//mismo
import { render, screen } from "@testing-library/react";
describe('Componente TextBox', () => {
  it('la caja se encuentra en el componente', () => {
    //Primero renderizamos el componente: se crea el DOM Virtual
    render(<TextBox />);
    //Buscamos en la pantalla del DOM Virtual un elemento según su Role.
    //Queremos buscar el role que sea 'caja' (role = 'caja' en el componente TextBox.jsx)
    //y que además su name sea 'es una caja'. El name se busca en el atributo title de
    //<Box>
    const box = screen.getByRole('caja', { name: 'es una caja' });
    // Finalmente comprobamos si la caja está en el documento con el matcher
    // toBeInTheDocument()
    expect(box).toBeInTheDocument();
  });
  it('el botón se encuentra en el componente', () => {
    render(<TextBox />);
    const button = screen.getByRole('button', { name: 'Pulsa para modificar el color' });
    expect(button).toBeInTheDocument();
  });
});
```

Aquí vemos que por cada test estamos renderizando varias veces lo mismo. Podemos poner la parte del render y de obtener los elementos de la pantalla dentro del `beforeEach(() => {})` tal y como vemos en el código de ejemplo que les he proporcionado.

Ejemplo 2.2: Simular la interacción del usuario con la interfaz: userEvent

<https://testing-library.com/docs/user-event/intro/>

Para simular la interacción del usuario con la interfaz se usa userEvent. Se debe importar primero:

```
import userEvent from "@testing-library/user-event";
```

Para usarlo dentro de un test debemos usar una función asíncrona. Vemos a continuación un ejemplo de test asíncrono con el userEvent en el que se simula que un usuario rellena un formulario que tiene un campo de texto y luego pica en el botón Enviar.

```
it('rellenar formulario', async () => {  
  //Suponemos que ya hemos renderizado el componente y hemos obtenido el campo de texto  
  //y el botón  
  //render (<Formulario />);  
  //txt = screen.getByLabelText('Nombre')  
  //button = screen.getByRole('button', name: {'Enviar'})  
  //Se inicializa el userEvent  
  const user = userEvent.setup()  
  
  //Después se simulan acciones del usuario. Ejemplo de acciones (hay que poner el await):  
  //El usuario borra el contenido del campo de texto txt.  
  await user.clear(txt)  
  
  //El usuario rellena un campo de texto con el texto 'Me muero de sueño en clase'  
  await user.type(txt, 'Me muero de sueño en clase')  
  
  //El usuario hace click en un botón.  
  await user.click(button)  
  
  //AQUÍ VENDRÍA LA PARTE DE TEST. Vamos a suponer que al picar en el botón se  
  //muestra un heading de nivel 3 en la interfaz.  
  expect(screen.getByRole('heading', { level: 3 })).toBeInTheDocument()  
});
```

Tabla resumen de algunas acciones que podemos simular con userEvent

Tabla resumen de algunas acciones que podemos simular con userEvent	
Inicialización del usuario	const user =userEvent.setup()
Borrar un campo de texto	user.clear(txt)
Escribir algo en un campo de texto	user.type(txt, 'hola')
Clicar en un botón	user.click(button)